

```

/**
 * startup-orchestrator.patch.ts
 *
 * Shows where each new service plugs into the existing StartupOrchestrator
 * staged startup. Do not replace the orchestrator – patch into it.
 *
 * Staged startup order (existing):
 *   Stage 1: Environment Validation
 *   Stage 2: Database Readiness          ← awaitDbReady()
 *   Stage 3: Production Account Cleanup
 *   Stage 4: YouTube Connection Health   ← auditTokensOnBoot() HERE
 *   Stage 5: Quota and Budget Recovery
 *   Stage 6: Queue Repair
 *   Stage 7: Resource Health
 *   Stage 8: Core Workers Start
 *   Stage 9: YouTube Safe Workers Start ← YouTube API workers only
 *   Stage 10: AI Scheduler Start
 *   Stage 11: Growth Engine Start
 *   Stage 12: Self-Healing Engine Start
 *   Stage 13: Perpetual Operating Loop
 */

// ——— IMPORTS — add to server/index.ts —————

import { auditTokensOnBoot }           from './services/token-guardian-hardened.js';
import { startStuckSchedulerRecovery,  from './services/stuck-scheduler-recovery.js';
      stopStuckSchedulerRecovery }
import { startDeadLetterDrain,         from './services/dead-letter-drain.js';
      stopDeadLetterDrain }

// Middleware wiring happens during Express setup (before stages):
// import { checkPromptInjection,
//         checkSqlInjection }         from './lib/prompt-injection-guard.js';

// ——— STAGE 4: YouTube Connection Health —————
//
// Find your existing Stage 4 handler and ADD auditTokensOnBoot():
//
//   async function stage4_youtubeConnectionHealth() {
//     // ... existing channel health checks ...
//
//     // ADD: Token null check and backup recovery
//     const tokenReports = await auditTokensOnBoot();
//     for (const report of tokenReports) {
//       if (report.status === 'null_no_backup') {
//         log.error(`[Startup] 🛑 ${report.channelName}: ${report.action}`);
//         // Channel is now marked needs_reconnect = true in DB.

```

```
//      // Publisher will refuse YouTube API calls for this channel.
//      } else {
//      log.info(`[Startup] ${report.channelName}: ${report.status} - ${report.action}`);
//      }
//  }
// }
```

// — STAGE 8: Core Workers Start —————

```
//
// Fix #1: Recovery workers belong here – NOT in Stage 9.
// They must run even when YouTube is disconnected or quota-blocked, because
// their job is to DEFER or ESCALATE stuck jobs safely, not to publish.
// Stage 9 is reserved for actual YouTube API workers only.
//
// Find your existing Stage 8 handler and ADD:
//
//  async function stage8_coreWorkersStart() {
//      // ... existing core worker starts ...
//
//      // ADD after CommandCenter is online:
//      startStuckSchedulerRecovery(); // defers/escalates stuck jobs safely
//      startDeadLetterDrain();       // Fix #2: void return – no await
//  }
```

// — STAGE 9: YouTube Safe Workers Start —————

```
//
// Stage 9 remains for actual YouTube API workers only.
// Do NOT add recovery services here.
//
//  async function stage9_youtubeSafeWorkersStart() {
//      // ... existing YouTube API workers (publisher, quota-aware uploader, etc.) ...
//      // Recovery services are already running from Stage 8 above.
//  }
```

// — GRACEFUL SHUTDOWN – handle both SIGTERM and SIGINT —————

```
for (const signal of ['SIGTERM', 'SIGINT'] as const) {
  process.on(signal, async () => {
    stopStuckSchedulerRecovery();
    stopDeadLetterDrain();
    // ... existing shutdown cleanup ...
  });
}
```

// — MIDDLEWARE WIRING (during Express setup, before stages) —————

```
//
```

```

// In your security middleware setup:
//
// app.use(async (req, res, next) => {
//   const clientId = req.ip ?? req.socket?.remoteAddress ?? '';
//
//   // Fix #1: check path + bounded query + bounded body – not just req.path
//   const sqlInput = [
//     req.path,
//     JSON.stringify(req.query ?? {}).slice(0, 2000),
//     JSON.stringify(req.body ?? {}).slice(0, 5000),
//   ].join(' ');
//
//   const sqlCheck = await checkSqlInjection(sqlInput, clientId);
//   if (sqlCheck.blocked) {
//     log.warn(`[Security] SQL injection blocked from ${clientId}`);
//     return res.status(400).end();
//   }
//
//   next();
// });
//
// In your AI content generation pipeline (wherever content passes through security):
//
// const injCheck = checkPromptInjection(generatedContent);
// if (injCheck.blocked && !injCheck.gamingFalsePositive) {
//   log.warn(`[Security] Prompt injection blocked: ${injCheck.label}`);
//   if (injCheck.severity === 'critical') {
//     throw new Error('Content blocked by injection guard');
//   }
// } else if (injCheck.gamingFalsePositive) {
//   log.debug(`[Security] Gaming false positive suppressed: ${injCheck.label}`);
// }

// ——— PRODUCTION GUARD – already at server/lib/production-guard.ts ———
//
// The real guard is a standalone file: server/lib/production-guard.ts
// It exports isProductionAutomationAllowed(userId, channelId?, platform?)
// returning { allowed: boolean; reason: string }.
//
// It is already imported in:
//   server/services/stuck-scheduler-recovery.ts
//   server/services/dead-letter-drain.ts
//
// Import it in any future background worker the same way:
// import { isProductionAutomationAllowed } from '../lib/production-guard.js';
//
// const guard = isProductionAutomationAllowed(userId, channelId, platform);
// if (!guard.allowed) {
//   log.debug(`[ServiceName] Skipping: ${guard.reason}`);

```

```

//     continue;
//   }

//
// Search the codebase for:
//   UPDATE channels SET access_token =
//   SET access_token =
//   db.update(channels).set({ accessToken:
//
// Replace each with:
//   import { writeTokensAtomic } from './services/token-guardian-hardened.js';
//
//   const result = await writeTokensAtomic(channelId, {
//     accessToken:    newAccessToken,
//     refreshToken:   newRefreshToken ?? undefined, // omit if not returned by Google
//     tokenExpiresAt: new Date(Date.now() + 3_600_000),
//   });
//   if (!result.success) {
//     log.error(`Token write failed for channel ${channelId}: ${result.error}`);
//     return;
//   }

// ——— PUBLISHER GUARD — before any YouTube API call —————
//
// import { channelCanPublish } from './services/token-guardian-hardened.js';
//
// Fix #2: channelCanPublish() returns { canPublish, shouldRefresh, reason }
// not a boolean — do NOT use `if (!await channelCanPublish(channelId))`.
//
//   async function publishToYouTube(channelId: number, ...) {
//     const guard = await channelCanPublish(channelId);
//
//     if (!guard.canPublish) {
//       if (guard.shouldRefresh) {
//         const refreshed = await refreshChannelToken(channelId); // your existing fn
//         if (!refreshed) {
//           log.warn(`[Publisher] Channel ${channelId} token refresh failed — blocking`);
//           return;
//         }
//         const afterRefresh = await channelCanPublish(channelId);
//         if (!afterRefresh.canPublish) {
//           log.warn(`[Publisher] Channel ${channelId} still blocked after refresh:
// ${afterRefresh.reason}`);
//           return;
//         }
//         // Token refreshed — fall through to YouTube API call
//       } else {
//         log.warn(`[Publisher] Channel ${channelId} blocked: ${guard.reason}`);

```

```

//         return; // Hard block – needs human reconnect
//     }
// }
//
//     // Safe to call YouTube API
// }

// ——— ACCEPTANCE CHECKLIST ———
//
// After wiring, verify these behaviours in the Replit logs:
//
// ✅ Prompt injection:
//   - "bypass the guard" in AC Valhalla description → debug log, NOT blocked
//   - "ignore all previous instructions" → blocked, severity critical
//   - GCP 35.191.x.x health checks → NOT flagged as SQL injection
//
// ✅ Token guardian:
//   - Boot log shows channel token health report for each YouTube channel
//   - If null: recovery attempt logged, then needs_reconnect if unrecoverable
//   - Token writes: "Tokens written atomically for channel X" in logs
//   - No token values in logs (only redacted prefix)
//
// ✅ Stuck scheduler:
//   - Fires every ~15 min, logs one summary line per cycle
//   - Items overdue 3+ hours with quota active → status 'deferred', not 'pending'
//   - After 5 misses → status 'escalated'
//   - Non-YouTube items → status 'permanent_fail'
//
// ✅ Dead letter drain:
//   - Fires every ~12 min, processes max 1 item
//   - Over 1 hour: max 5 items requeued (not 25)
//   - Stale items (>30d) marked 'expired' once on first run
//   - Demo/test/phantom user items are never requeued
//
// Run typecheck: npx tsc --noEmit
// Run build:     npm run build
// Run tests:     npm test (if test suite exists)

```